

Software Requirements Specification

for

AI Chatbot for College Helpdesk

Version 1.0 approved

Prepared by:

Department of Computer Science & Engineering

Semester 5 Section A

S. No.	Name	Roll Number
1	Anupriya Saxena	202401100200070
2	Anurag Srivastava	202401100200071
3	Anushka Kesarwani	202401100200072
4	Anushka Sharma	202401100200073

Table of Contents

1. Introduction ..... Error! Bookmark not defined.
1.1 Purpose ..... **Error! Bookmark not defined.**
1.2 Document Conventions ..... **Error! Bookmark not defined.**
1.3 Intended Audience and Reading Suggestions ..... **Error! Bookmark not defined.**
1.4 Product Scope ..... **Error! Bookmark not defined.**
1.5 References ..... **Error! Bookmark not defined.**
2. Overall Description ..... Error! Bookmark not defined.
2.1 Product Perspective ..... **Error! Bookmark not defined.**
2.2 Product Functions ..... **Error! Bookmark not defined.**
2.3 User Classes and Characteristics ..... **Error! Bookmark not defined.**
2.4 Operating Environment ..... **Error! Bookmark not defined.**
2.5 Design and Implementation Constraints ..... **Error! Bookmark not defined.**
2.6 User Documentation ..... **Error! Bookmark not defined.**
2.7 Assumptions and Dependencies ..... **Error! Bookmark not defined.**
3. External Interface Requirements ..... Error! Bookmark not defined.
3.1 User Interfaces ..... **Error! Bookmark not defined.**
3.2 Hardware Interfaces ..... **Error! Bookmark not defined.**
3.3 Software Interfaces ..... **Error! Bookmark not defined.**
3.4 Communications Interfaces ..... **Error! Bookmark not defined.**
4. System Features ..... Error! Bookmark not defined.
4.1 System Feature 1 ..... **Error! Bookmark not defined.**
4.2 System Feature 2 (and so on) ..... **Error! Bookmark not defined.**
5. Other Nonfunctional Requirements ..... Error! Bookmark not defined.
5.1 Performance Requirements ..... **Error! Bookmark not defined.**
5.2 Safety Requirements ..... **Error! Bookmark not defined.**
5.3 Security Requirements ..... **Error! Bookmark not defined.**
5.4 Software Quality Attributes ..... **Error! Bookmark not defined.**
5.5 Business Rules ..... **Error! Bookmark not defined.**
6. Other Requirements ..... Error! Bookmark not defined.
Appendix A: Glossary ..... Error! Bookmark not defined.
Appendix B: Analysis Models ..... Error! Bookmark not defined.
Appendix C: To Be Determined List ..... Error! Bookmark not defined.

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements for the AI Chatbot for College Helpdesk, Version 1.0. The chatbot is a web- and mobile-accessible conversational assistant intended to automatically answer student, faculty, and visitor queries related to admissions, examinations, fees, timetables, hostel facilities, and other general college services. This document covers the complete scope of the chatbot system, including its interaction with the college's existing student database and helpdesk ticketing workflow.

1.2 Document Conventions

This document follows the IEEE SRS format. Requirement statements are uniquely identified using the prefix REQ-<Feature Number>.<Sequence Number> (e.g., REQ-1.1). Priorities are indicated as High, Medium, or Low next to each system feature; unless otherwise stated, a feature's priority is inherited by all its child requirements. The keywords "shall" and "must" indicate a mandatory requirement, while "should" indicates a desirable but non-mandatory requirement.

1.3 Intended Audience and Reading Suggestions

This document is intended for the project guide/evaluator, the development team (students), the college IT administration, and end users such as students and faculty. Developers should read Sections 2 through 5 in detail to understand system behaviour before implementation. Project evaluators and the college administration may focus on Sections 1, 2, and 4 to understand scope and functionality. Testers should refer to Section 4 (System Features) and Section 5 (Nonfunctional Requirements) to design test cases.

1.4 Product Scope

The AI Chatbot for College Helpdesk aims to reduce the workload on the college's administrative and helpdesk staff by automatically resolving frequently asked and repetitive queries through natural language understanding. The chatbot will be available 24x7 through a website widget and an optional mobile interface, will authenticate registered students for personalized responses (e.g., attendance, fee dues), and will escalate unresolved or complex queries to a human helpdesk operator by automatically raising a support ticket. The long-term goal is to reduce average query response time, reduce the manual load on helpdesk staff, and improve student satisfaction with support services.

1.5 References

- IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications.
- College Examination and Academic Policy Manual, <Institution Name>, <Year>.
- Existing Helpdesk Ticketing System documentation, <Institution IT Department>.
- Dialogflow / Rasa Open Source documentation (candidate NLP engines).
- Web Content Accessibility Guidelines (WCAG) 2.1.

2. Overall Description

2.1 Product Perspective

The AI Chatbot for College Helpdesk is a new, self-contained software product that integrates with the college's existing systems rather than replacing them. It will interface with the Student Information System (SIS) / ERP for academic and fee data, and with the existing manual/email-based helpdesk process for ticket escalation. The chatbot sits as a front-end conversational layer on top of these systems: students interact with the chatbot, which in turn queries backend services or creates tickets on the student's behalf when it cannot resolve a query directly.

At a high level, the system consists of four major components: (1) a Chat Interface (web widget and optional mobile screen), (2) an NLP/Dialogue Engine that classifies intent and extracts entities, (3) a Backend/Integration Layer that talks to the college database and ticketing system, and (4) an Admin Dashboard used by helpdesk staff to monitor conversations, manage the knowledge base, and handle escalated tickets.

2.2 Product Functions

- Understand natural-language student queries and respond with relevant information (FAQ-style and personalized).
- Authenticate students (login/OTP) to provide personalized answers such as fee due date, attendance percentage, and exam hall ticket status.
- Handle common helpdesk categories: admissions, examinations, fee payment, timetable, hostel, library, and placement queries.
- Escalate unresolved or low-confidence queries to a human agent by automatically creating a support ticket with conversation context.
- Allow helpdesk staff/admins to review conversations, update the knowledge base/FAQ, and track ticket status via an admin dashboard.
- Provide conversation analytics (most-asked questions, unresolved query rate, average response time) to administrators.
- Support multi-turn conversation (follow-up questions) and basic conversational chit-chat/greetings.

2.3 User Classes and Characteristics

User Class	Characteristics
Student (Guest)	Not logged in; can ask general FAQ-type questions only (admission process, general timetable, contact info).
Student (Authenticated)	Logged in with college ID; can query personal academic/fee data; most frequent user class; varying technical expertise.
Faculty/Staff	May query internal circulars, room bookings, or general administrative information; moderate frequency of use.
Helpdesk Admin/Operator	Manages knowledge base, monitors chats, resolves escalated tickets; requires dashboard access with elevated privileges.
System Administrator	Configures integrations, manages user roles, monitors system health and logs; highest privilege level, least frequent use.

2.4 Operating Environment

- Client side: any modern web browser (Chrome, Firefox, Edge, Safari) on desktop or mobile; responsive web widget embeddable in the existing college website.
- Server side: cloud-hosted or on-campus server running the NLP engine, application backend, and database (Linux-based, containerized deployment recommended).
- Database: relational database (e.g., MySQL/PostgreSQL) for chat logs, tickets, and FAQ knowledge base; integration with existing college ERP/SIS via REST API.
- NLP Engine: an intent-classification framework such as Rasa Open Source, Dialogflow, or a fine-tuned transformer-based model.

2.5 Design and Implementation Constraints

- Must integrate with the college's existing ERP/SIS through its available APIs; direct database access to third-party systems is not permitted.
- Must comply with the institution's data privacy and IT security policies for handling student personal and academic data.
- Development timeline and resources are constrained by the academic project schedule (student project team, limited budget).
- The system must operate within the free/low-cost tiers of chosen NLP and hosting services where applicable, due to budget constraints.
- Front end must be responsive and accessible on low-bandwidth campus Wi-Fi connections.

2.6 User Documentation

The following documentation will be delivered along with the software: a Student User Guide (how to interact with the chatbot, login, and escalate a query), an Admin Manual (managing the knowledge base, dashboard usage, ticket handling), and in-app contextual help (a short "How can I help you?" onboarding message and sample query suggestions shown on first use).

2.7 Assumptions and Dependencies

- It is assumed that the college's ERP/SIS exposes (or will expose) a REST API for fetching student academic and fee information.
- It is assumed that helpdesk staff will be available to handle tickets escalated by the chatbot during working hours.
- The project depends on a third-party or open-source NLP/intent-classification engine for language understanding.
- It is assumed that an initial FAQ/knowledge base dataset (commonly asked questions and answers) will be provided by the college administration for training the chatbot.

3. External Interface Requirements

3.1 User Interfaces

- A chat widget embedded in the bottom-right corner of the college website, expandable into a full chat window.
- A welcome message with 3–4 quick-reply suggestion buttons (e.g., "Fee Details", "Exam Schedule", "Talk to a human") shown when the chat opens.
- A text input box with send button, and support for quick-reply buttons/cards for structured answers (e.g., FAQ menus).
- A login prompt (college ID + password/OTP) triggered only when a query requires personalized/authenticated data.
- An "Escalate to human agent" option visible at all times, and a ticket confirmation message (with ticket ID) shown when escalation occurs.
- Admin Dashboard: a web-based interface with sections for Live Chats, Tickets, Knowledge Base/FAQ Editor, and Analytics, following the college's standard UI style guide where one exists.

3.2 Hardware Interfaces

The chatbot does not directly interface with any specialized hardware. It requires standard client devices (desktop/laptop/smartphone) with a web browser and internet connectivity, and standard server hardware (physical or cloud virtual machine) capable of hosting the application and NLP services.

3.3 Software Interfaces

Interface	Description
College ERP/SIS	REST API integration to fetch/verify student data such as roll number, fee status, attendance, and exam hall ticket availability.
NLP/Intent Engine	Rasa Open Source / Dialogflow (or equivalent) for intent classification, entity extraction, and dialogue management.
Helpdesk Ticketing System	Existing or newly built ticketing module used to log, assign, and track escalated queries.
Database	MySQL/PostgreSQL for storing chat logs, FAQ knowledge base, user sessions, and ticket records.
Notification Service	Email/SMS gateway (e.g., SMTP, Twilio) used to notify students of ticket status updates.

3.4 Communications Interfaces

- All client-server communication shall use HTTPS with TLS 1.2 or higher.
- The chatbot backend shall expose/consume RESTful APIs (JSON format) for all integrations with the ERP/SIS and ticketing system.
- Email notifications shall be sent via SMTP; optional SMS notifications via a third-party SMS gateway API.
- WebSocket (or long-polling as a fallback) shall be used between the chat widget and backend for real-time message delivery.

4. System Features

This section organizes the functional requirements of the chatbot by major system feature. Each feature includes its description and priority, a typical stimulus/response sequence, and its detailed functional requirements.

4.1 Natural Language Query Understanding

4.1.1 Description and Priority

Allows the chatbot to interpret free-text student queries, classify their intent (e.g., "fee due date", "exam schedule"), and extract relevant entities (e.g., semester, subject code). Priority: High.

4.1.2 Stimulus/Response Sequences

The student types a question in natural language → the system classifies the intent and extracts entities → the system retrieves the matching answer or data → the system displays the response, optionally with follow-up quick-reply options.

4.1.3 Functional Requirements

REQ-1.1: The system shall classify incoming student messages into a predefined set of intents with a confidence score.

REQ-1.2: The system shall extract relevant entities (e.g., date, subject, semester, department) from the query text.

REQ-1.3: If the intent confidence score is below a configurable threshold (default 60%), the system shall ask a clarifying question instead of guessing.

REQ-1.4: The system shall support multi-turn conversations, retaining context (e.g., selected semester) for up to 5 follow-up messages.

REQ-1.5: The system shall support both English and at least one regional language, if configured by the administrator.

4.2 FAQ and Knowledge Base Handling

4.2.1 Description and Priority

Provides answers to frequently asked, non-personalized questions (admissions process, general timetable, hostel rules, library timings, etc.) drawn from an administrator-managed knowledge base. Priority: High.

4.2.2 Stimulus/Response Sequences

The student asks a general question → the system matches it to a knowledge base entry → the system returns the stored answer, with an option to mark the answer as "helpful" or "not helpful".

4.2.3 Functional Requirements

REQ-2.1: The system shall maintain a searchable knowledge base of question-answer pairs categorized by topic (admissions, exams, fees, hostel, library, placements, general).

REQ-2.2: The system shall allow administrators to add, edit, or delete knowledge base entries through the admin dashboard without requiring code changes.

REQ-2.3: The system shall present a "Was this helpful?" feedback option after each FAQ answer and log the response.

REQ-2.4: The system shall log queries that could not be matched to any knowledge base entry, for administrator review.

4.3 Personalized/Authenticated Queries

4.3.1 Description and Priority

Allows logged-in students to query personal academic and administrative data such as fee dues, attendance percentage, and exam hall ticket status. Priority: High.

4.3.2 Stimulus/Response Sequences

The student asks a personalized question (e.g., "What is my fee due date?") → if not logged in, the system prompts for login/OTP → upon successful authentication, the system queries the ERP/SIS API → the system displays the personalized result.

4.3.3 Functional Requirements

REQ-3.1: The system shall prompt for authentication (college ID and password, or OTP) whenever a query requires personal/academic data.

REQ-3.2: The system shall retrieve student-specific data via the ERP/SIS API and shall not store duplicated academic records beyond the active session, unless explicitly cached with administrator approval.

REQ-3.3: The system shall time out an authenticated session after 15 minutes of inactivity and require re-authentication for further personalized queries.

REQ-3.4: The system shall display a clear error message if the ERP/SIS API is unavailable, and shall offer to escalate the query to a human agent.

4.4 Ticket Escalation to Human Agent

4.4.1 Description and Priority

Automatically or manually escalates unresolved, low-confidence, or explicitly requested queries to a human helpdesk operator by creating a support ticket. Priority: High.

4.4.2 Stimulus/Response Sequences

The chatbot fails to resolve a query, or the student clicks "Talk to a human" → the system collects/confirms the student's contact details and query summary → the system creates a ticket in the helpdesk system → the system displays a ticket ID and expected response time to the student.

4.4.3 Functional Requirements

REQ-4.1: The system shall automatically offer escalation when it cannot classify an intent with sufficient confidence after two clarifying attempts.

REQ-4.2: The system shall allow the student to explicitly request a human agent at any point in the conversation.

REQ-4.3: The system shall create a ticket containing the student's ID/contact info, the conversation transcript, and a suggested category, and shall return a unique ticket ID to the student.

REQ-4.4: The system shall notify the student via email/SMS when the status of their ticket changes (e.g., assigned, resolved).

REQ-4.5: The system shall route tickets to the appropriate helpdesk department based on the query category (examinations, fees, hostel, etc.).

4.5 Admin Dashboard and Knowledge Base Management

4.5.1 Description and Priority

Provides helpdesk staff and administrators with tools to monitor live/past conversations, manage the FAQ knowledge base, and handle escalated tickets. Priority: Medium.

4.5.2 Stimulus/Response Sequences

The admin logs into the dashboard → the admin views a list of open tickets and recent unresolved queries → the admin responds to a ticket or updates the knowledge base → the system reflects the update in the chatbot's future responses.

4.5.3 Functional Requirements

REQ-5.1: The system shall provide role-based access control distinguishing System Administrator and Helpdesk Operator roles.

REQ-5.2: The system shall allow operators to view, filter, and update the status of tickets (Open, In Progress, Resolved, Closed).

REQ-5.3: The system shall allow administrators to review the list of unmatched/unresolved queries and convert them into new knowledge base entries.

REQ-5.4: The system shall display basic analytics: total conversations, top 10 asked questions, escalation rate, and average resolution time, filterable by date range.

4.6 Multi-Channel Access

4.6.1 Description and Priority

Allows students to access the chatbot from multiple entry points: the college website widget and, optionally, a mobile-friendly interface. Priority: Low.

4.6.2 Stimulus/Response Sequences

The student opens the chat widget on the college website or mobile browser → the system loads the conversation interface and, if available, resumes the previous session.

4.6.3 Functional Requirements

REQ-6.1: The chat widget shall be responsive and usable on both desktop and mobile browser viewports.

REQ-6.2: The system should persist conversation history for a logged-in student across sessions for up to 30 days.

REQ-6.3: The system may optionally integrate with a messaging platform (e.g., WhatsApp/Telegram) as a future enhancement; this is out of scope for Version 1.0.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

- The system shall respond to at least 90% of FAQ-type queries within 2 seconds under normal load.
- The system shall support at least 200 concurrent chat sessions without degradation in response time, to accommodate peak periods (e.g., admission or exam season).
- Ticket creation upon escalation shall complete within 3 seconds of the student confirming escalation.

5.2 Safety Requirements

The system does not control any physical hardware and therefore poses no direct physical safety risk. However, the system shall avoid providing definitive academic or legal advice (e.g., final exam eligibility decisions) without clearly indicating that such decisions are subject to confirmation by the concerned department, to prevent students from acting on incorrect automated information.

5.3 Security Requirements

- All communication between the client and server shall be encrypted using HTTPS/TLS.
- Student authentication credentials shall never be stored in plain text; passwords shall be hashed, and OTPs shall expire after 5 minutes.
- Access to the admin dashboard shall require role-based authentication, and all administrative actions shall be logged for audit purposes.
- The system shall comply with the institution's data protection policy regarding storage and retention of student personal data, and shall allow chat logs to be purged after a configurable retention period.

5.4 Software Quality Attributes

- Usability: A first-time user should be able to get a relevant answer to a common question within 2 interactions, without training.
- Availability: The chatbot service shall target 99% uptime during college working hours (8 AM – 8 PM, Monday–Saturday).
- Maintainability: Knowledge base updates shall not require code redeployment or developer involvement.
- Scalability: The architecture shall allow horizontal scaling of the NLP/backend service to handle increased query volume.
- Portability: The chat widget shall be embeddable in any standard HTML-based website with minimal integration effort (a single script tag).

5.5 Business Rules

- Only authenticated students may access personal academic/fee data; guest users are restricted to general FAQ-type queries.

- Only Helpdesk Operators and System Administrators may modify the knowledge base or view/manage tickets.
- A ticket that remains unresolved for more than 48 working hours shall be automatically flagged as "overdue" and escalated to a senior operator.

6. Other Requirements

Internationalization: the system should be designed to allow addition of further regional languages in future versions without major architectural changes. Legal: the system shall include a visible privacy notice informing users what data is collected and how it is used, in line with the institution's data protection policy. Reuse: where possible, the project shall reuse the college's existing authentication service rather than implementing a parallel login system.

Appendix A: Glossary

Term	Definition
ERP/SIS	Enterprise Resource Planning / Student Information System — the college's existing academic and administrative database system.
Intent	The purpose or goal behind a user's message, as classified by the NLP engine (e.g., "check_fee_status").
Entity	A specific piece of information extracted from a user's message, such as a date, semester, or subject code.
NLP	Natural Language Processing — the field of AI concerned with understanding and generating human language.
Ticket	A record created in the helpdesk system representing a query escalated to a human operator for resolution.
OTP	One-Time Password — a temporary code used for verifying a student's identity during login.

Appendix B: Analysis Models

This appendix would normally include supporting diagrams such as: a Data Flow Diagram showing how a student query moves from the chat widget through the NLP engine, backend, and ERP/SIS; a Use Case Diagram covering actors (Guest Student, Authenticated Student, Helpdesk Operator, Administrator) and their interactions with the system; and an Entity-Relationship Diagram for the chat logs, knowledge base, and ticket database. These diagrams should be created using a tool such as draw.io, Lucidchart, or a UML tool and attached/embedded here in the final submission.

Appendix C: To Be Determined List

TBD No.	Description	Section Reference
TBD-1	Final choice of NLP engine (Rasa vs. Dialogflow vs. custom transformer model) pending feasibility evaluation.	Section 2.4
TBD-2	Exact ERP/SIS API endpoints and authentication mechanism, pending confirmation from the college IT department.	Section 3.3
TBD-3	List of regional languages to be supported, pending stakeholder input.	Section 4.1
TBD-4	Final SLA for ticket resolution time, pending discussion with helpdesk staff.	Section 5.5